End-to-End Machine Learning Project

Part 6: prepare the data – feature scaling and transformation

Marcus Birkenkrahe

COSC 3430 · Artificial Intelligence · Augustana University

Based on Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch*, Ch. 2

Fall 2026

Roadmap

Ten steps, over several code-along lectures:

1. Look at the big picture
2. Get the data
3. Explore and visualize the data
4. Create the test data
5. Prepare the data for learning
6. Select a model and train it
7. Fine-tune the model
8. Present your solution
9. Launch the production system
10. Monitor and maintain it

Today: step 5, second part – feature scaling and transformation.
Next time: transformation pipelines and custom transformers.

Review: where part 5 left off

1. What did we split the stratified training set into – and which part holds only the numbers?
   → Predictors `housing` and labels `housing_labels`; the numerical features in `housing_num`
2. `total_bedrooms` has 168 missing values. How did we fill them?
   → `SimpleImputer` with the median
3. How did we turn the text attribute `ocean_proximity` into numbers?
   → `OneHotEncoder`: one binary column per category
4. What is the job of `fit()`, and what is the job of `transform()`?
   → `fit()` learns parameters from the training data, `transform()` applies them

Question

Algorithms that compute distances across features – regression, kNN, neural networks – care about scales.

Is this relevant for the housing data?

How would you find out?

Compare the spread of two features

→ **Switch to editor · code-along 1**

```
print(housing.describe()[
    ["total_rooms", "median_income"]])
```

→ total_rooms lies in $[2, 39320]$, median_income in $(0, 15]$: the scales are vastly different.

A feature with range 0–40,000 dominates a feature with range 0–15, independent of its predictive importance.

	total_rooms	median_income
count	16512.000000	16512.000000
mean	2639.402798	3.870428
std	2185.287466	1.891936
min	2.000000	0.499900
25%	1447.000000	2.562500
50%	2125.000000	3.538500
75%	3154.000000	4.750000
max	39320.000000	15.000100

Two common ways to get all features onto one scale

1. **Min-max scaling** ("normalization"): shift and rescale all values so they range from 0 to 1

$$x' = \frac{x - \min}{\max - \min}$$

   Strongly affected by outliers.

2. **Standardization** ("z-score normalization"): mean 0, standard deviation 1, range not restricted

$$z = \frac{x - \mu}{\sigma}$$

Should scalers be fit to all the data, the training data, or the test data?

→ Only to the training data – otherwise the test set leaks into the model. A trained scaler can then `transform()` any other set.

Min-max scaling: `MinMaxScaler`

→ **Switch to editor · code-along 2**

```
from sklearn.preprocessing import MinMaxScaler

min_max_scaler = MinMaxScaler(
    feature_range=(-1, 1))
housing_num_min_max_scaled = \
    min_max_scaler.fit_transform(housing_num)

col = "total_rooms"
idx = housing_num.columns.get_loc(col)
before = housing_num[col].describe()
after = pd.Series(
    housing_num_min_max_scaled[:, idx]).describe()
print("Before:\n", before)
print("After:\n", after)
```

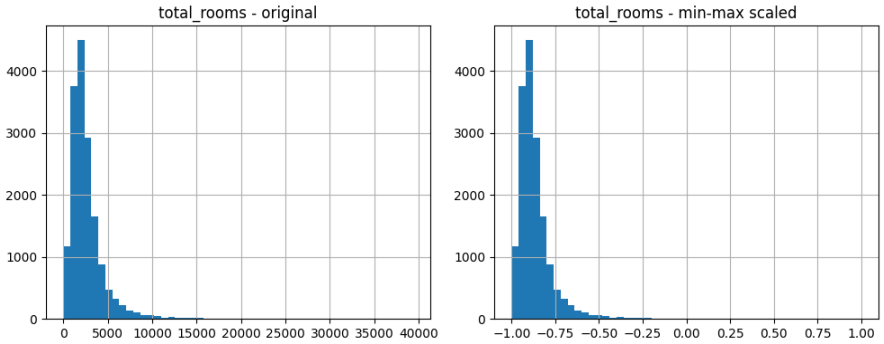
```
After:
 count    16512.000000
mean        -0.865842
std          0.111160
min         -1.000000
25%         -0.926497
50%         -0.892009
75%         -0.839666
max          1.000000
```

`Before` is the `total_rooms` column from slide 5.

`feature_range` sets the target interval – here

Min-max scaling changes the scale, not the shape

→ **Switch to editor · code-along 3**



Three quarters of the districts are squeezed into $[-1, -0.84]$: one district with 39,320 rooms takes the value 1.

```
pd.Series(housing_num_min_max_scaled[:, idx]).hist(ax=axes[1], bins=50)
```

Question

Suppose one district has a median income of 100 – by mistake – instead of 0–15.

What does min-max scaling do with it?

And with all the other districts?

Standardization is much less affected by outliers

- **Min-max**: the outlier is mapped to 1 – all other values are squeezed into $[0, 0.15]$ and become irrelevant
- **Standardization**: the outlier shifts the mean slightly, but the z-scores of all the other values remain almost unchanged
- Standardization is concerned with the overall spread, not with the two most extreme values

Standardization: StandardScaler

→ **Switch to editor · code-along 4**

```
from sklearn.preprocessing import StandardScaler

std_scaler = StandardScaler()
housing_num_std_scaled = \
    std_scaler.fit_transform(housing_num)

after = pd.Series(
    housing_num_std_scaled[:, idx]).describe()
print("After:\n", after)
```

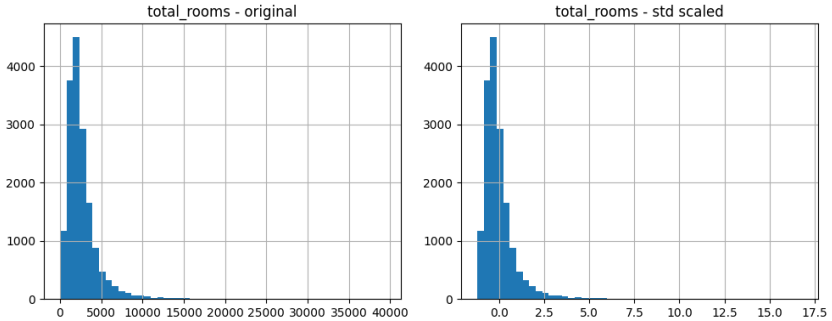
```
After:
count    1.651200e+04
mean    -5.916886e-17
std      1.000030e+00
min     -1.206927e+00
25%     -5.456668e-01
50%     -2.354008e-01
75%      2.354897e-01
max      1.678576e+01
```

What do you notice?

- → The mean is essentially 0, the standard deviation essentially 1
- → The max sits 16 standard deviations above the mean: the district with 39,320 rooms – scaled, not removed

A heavy tail

→ **Switch to editor · code-along 5**



The spread between the mean and the max is huge: a **heavy tail**. Scaling does not change that – a **transformation** that shrinks the tail and makes the distribution more symmetric does.

Heavy tails: population

→ **Switch to editor · code-along 6**

```
print(housing_num["population"]
      .describe())
```

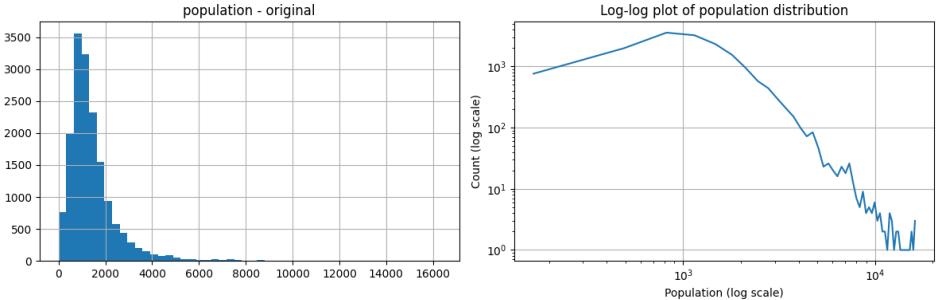
```
count    16512.000000
mean      1425.513929
std       1094.795467
min          3.000000
25%        787.000000
50%       1167.000000
75%       1726.000000
max      16305.000000
```

To shrink a heavy tail, replace the feature with

- its square root, $\sqrt{x}$
- a power between 0 and 1, x^p
- its logarithm, $\log x$

A power law

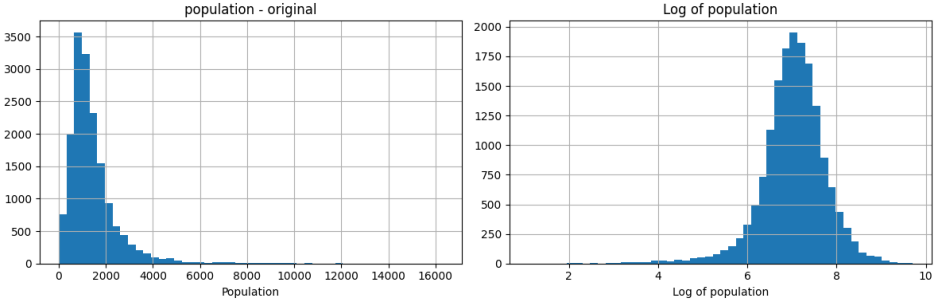
→ **Switch to editor · code-along 7**



- The histogram drops off slowly to the right – not exponentially
- In a log-log plot, a power law shows as a straight line: a hump around 1,000 (most districts), then a linear drop-off to 10,000

Log transform

→ **Switch to editor · code-along 8**



After the log transform, the distribution is roughly bell-shaped.

Bucketizing, and multimodal distributions

- **Bucketizing**: cut the distribution into roughly equal-sized buckets and replace the value with the bucket index (as in stratified sampling)
  - The result is roughly uniform – no further scaling needed
- **Multimodal** distributions, e.g. `housing_median_age`:
  - treat the bucket IDs as categories (then encode them, e.g. one-hot), or
  - add one feature per mode: the **similarity** between the value and that mode, e.g. houses aged 35 years
- Similarity is measured with a **radial basis function** (RBF): it depends only on the distance between the input value and a fixed point

Gaussian RBF: similarity to age 35

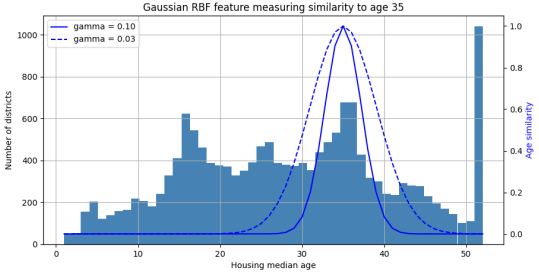
→ **Switch to editor · code-along 9**

$$\text{similarity}(x) = e^{-\gamma\,(x-35)^2}$$

γ sets how quickly the similarity decays as x moves away from 35.

```
from sklearn.metrics.pairwise \
    import rbf_kernel

age_simil_35 = rbf_kernel(
    housing[["housing_median_age"]],
    [[35]], gamma=0.1)
```



The new feature peaks at 35, at the spike in the age distribution. If this age group correlates with lower prices, the feature may help.

The target, and transformations you write yourself

- The **target** values may need a transformation too, if they have a heavy tail
- If you log-transform the target, the model predicts the *log* of the median house value
- Most scikit-learn transformers have an `inverse_transform()` method to get back to the original scale
- If the transformation you need does not exist, write your own **custom transformer** – next time

What we transformed

- **Scales**: features with very different ranges, e.g. `total_rooms` vs. `median_income`, distort distance-based algorithms
- **Min-max scaling** (`MinMaxScaler`): fixed range, strongly affected by outliers
- **Standardization** (`StandardScaler`): mean 0, standard deviation 1, robust to single outliers
- **Fit scalers to the training data only**, then transform any set
- **Heavy tails**: square root, power, or log – `population` follows a power law and is bell-shaped after a log transform
- **Bucketizing** and **RBF similarity** (`rbf_kernel`) for multimodal features

Next: transformation pipelines and custom transformers.